Exceptions and Testing

CSCI 1101

2024-04-04

Exception Handling Review

Last time we met python's exception-handling construct, the **try statement**:

```
try :
    <try_block>
except <exception_type_1> :
    <handle_exception_type_1>
 ⋮
except <exception_type_n> :
    <handle_exception_type_n>
```

where exceptions that are raised within the `try` cluase may be handled by an `except` clause.

We can handle multiple exception types in the same way by listing them in the same clause. Listing no exception types handles all exceptions.

What Could Go Wrong?

Consider the following function to parse a list of numbers from a string:

```
def parse_numbers(text) :
  """ sig: (str) --> list int """
  nums = []
  for part in text.split(",") :
    nums.append(int(str.strip(part)))
  return nums
```

Suppose we want to use it to get a certain number of numbers from the user:

```
def get_numbers(n) :
  text = input(f"enter {n} comma-separated numbers: ")
  return parse_numbers(text)
```

WCGW?

Handling Exceptions

We can fix this by putting the code that could raise an exception in a `try` clause and handling the exception:

```
def get_numbers(n) :
  while True :
    try :
      text = input(f"enter {n} comma-separated numbers: ")
      return parse_numbers(text)
    except ValueError :
      print(f"I couldn't understand '{text}'")
```

At least this doesn't crash.

But it might not give the right number of numbers.

Raising Exceptions

We can raise our own exceptions to indicate that something has gone wrong using the `raise` keyword:

```
def get_numbers(n) :
  while True :
    try :
      text = input(f"enter {n} comma-separated numbers: ")
      numbers = parse_numbers(text)
      if len(numbers) != n :
        raise ValueError("Wrong number of numbers!")
      return numbers
    except ValueError:
      print(f"'{text}' is not {n} numbers")
```

Assertions

Assertions give us a convenient way to signal that something has gone wrong if some condition fails:

```
assert <boolean_condition> , <error_string>
```

is shorthand for:

```
if not <boolean_condition> :
    raise AssertionError(<error_string>)
```

For example:

```
assert 1+1 == 2 , "I broke math"
assert False == True , "Look at me: I'm a politician"
```

Activity: Assertions

Rewrite the `get_numbers` function using an assertion to make sure that we have the right number of numbers.

Binding Exceptions

We can gain access to the actual `exception`, not just its *type* by binding it in an `except` clause:

```
except <exception_type> as <name> :
```

This can be useful if we want to analyze or print what went wrong.

Getting Numbers (Final Version)

For example:

```
def get_numbers(n) :
  while True :
    try :
      text = input(f"enter {n} comma-separated numbers: ")
      numbers = parse_numbers(text)
      assert len(numbers) == n , "Wrong number of numbers!"
      return numbers
    except ValueError :
      print(f"I couldn't understand '{text}'")
    except AssertionError as err :
      print(err)
```

Activity: Getting a Number in a Range

Complete the following function definition:

```
def get_num_between(low , high) :
    '''
    sig : (int , int) --> int
    prompts the user for a number between low and high.
    '''
```

For example:

```
> get_num_between(1 , 10)
enter a number between 1 and 10: eleven
I couldn't understand 'eleven'
enter a number between 1 and 10: 11
11 is not between 1 and 10
enter a number between 1 and 10: 10
10
```

Testing

As our programs get more complex it becomes harder to know that we haven't made a mistake.

One way to try to find mistakes is to *test* your functions.

To test a function you give it input for which you know *independently* what the result should be.

Then you compare the observed result with the expected result.

Example: Testing Leap Year

Recall the leap year deciding function we wrote in week 3:

```
def leap_year(y) :
    return (y%4 == 0 and y%100 != 0) or y%400 == 0
```

Independently, we know that:

- 2000 was a leap year (it is divisible by 400),
- 1900 was not a leap year (it is divisible by 100 but not 400),
- 2024 is a leap year (it's divisible by 4 but not 100),
- 2023 was not a leap year (it's not divisible by 4).

We can use these as **test cases**:

```
def test_leap_year() :
  assert leap_year(2000) and not leap_year(1900) \
  and leap_year(2024) and not leap_year(2023)
```

Guidelines for Writing Tests

A programs should conform to its **specification**, a (formal or informal) description of the intended behavior. When writing test cases, consider:

- where we know/infer case distinctions occur,
- where we know/infer iteration occurs,
- where we know/infer exception handling occurs.

We want to test each of the possible “code paths” to see that it conforms to the specification. Additionally, we want to test “edge cases”, including:

- empty or singleton lists/strings,
- zero or negative numbers,
- dictionaries in which a key is not found, or where a value has multiple keys.

Writing tests is as much an art as a science. You will get better with experience.

The Role of Testing

Writing individual tests this way is not a panacea;

scalability: it is slow and tedious,

correctness: now we need to write correct tests as well as correct code to test,

coverage: we write just a few tests out of a multitude of possibilities,

blindspots: the same oversights that cause buggy code tend to cause tests that fail to detect that buggy code,

complexity: many bugs are not caused by an error *within* one component, but by the interactions *between* components.

Testing can detect only the *presence* of bugs, not the *absence* of bugs.

To Do

Finish *Project 3 - Automated Grader (Part 1)* on CodeRunner by midnight this Friday.

Study for midterm 2 next Tuesday (2024-04-09).

Finish *Homework 9: Exceptions* on CodeRunner by 11:00AM next Thursday.